

New CMS (DM)WM: Discussion with HTCondor

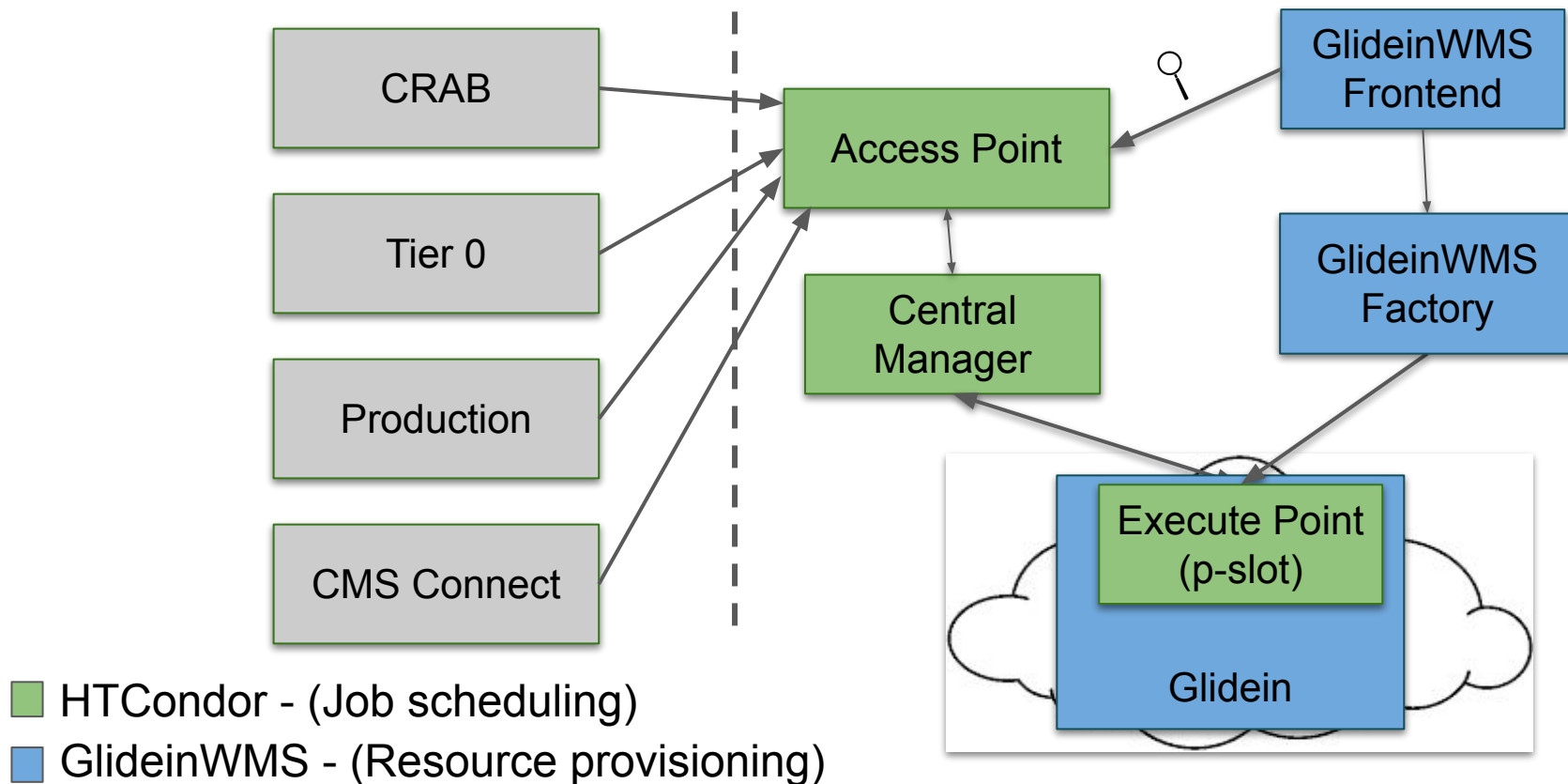
Marco Mascheroni and Andrea Piccinelli

Why this meeting?

- Ongoing investigations to understand what is the more sustainable effort to develop the CMS WMS for HL-LHC (see details in next slides)
 - Known limitations from the current CMS WMS that requires change of design principles
 - In the scenario we build our own solution, some architectural choices need to be taken upfront
 - Coordination between CMS WMS and HTCondor/GlideInWMS about architectural choices
 - Suggestions about how to interact with HTCondor
 - Features that HTCondor team could develop to support CMS choices
 - Make efficient architectural choices based on HTCondor expert recommendations
 - Interest in future closer collaboration between CMS and HTCondor dev teams

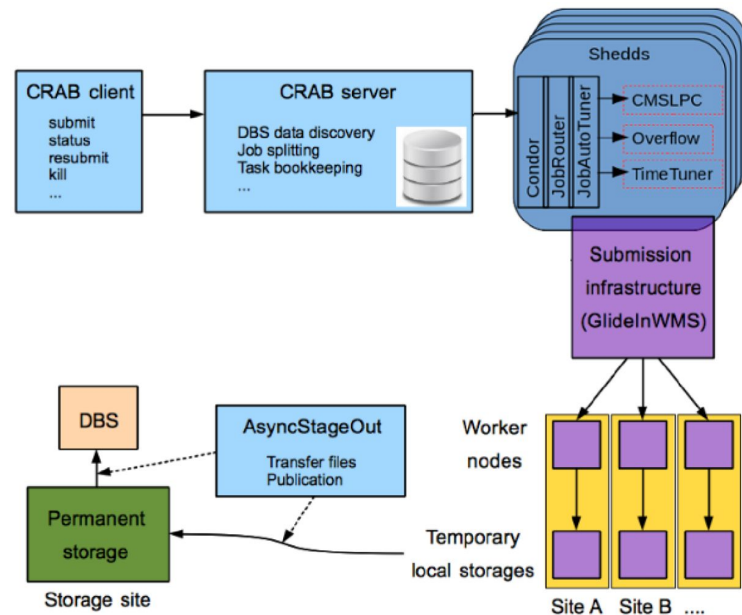
Workflows

Jobs (SI)



CRAB

- CRAB is the CMS tool that allows **physicists** to submit, manage, and monitor distributed user analysis jobs
- Mostly **used for**:
 - Skimming, slimming, and ntuple production from AOD/MiniAOD/NanoAOD
 - Parameter scans and systematic variations
 - Creating Private MonteCarlo Samples
- **CLI** to manage workflows
 - Submission, Data discovery, Job splitting, Status, Resubmission
 - **Output management** (from worker node to final destination)



Tier0

<https://cmst0.docs.cern.ch/>

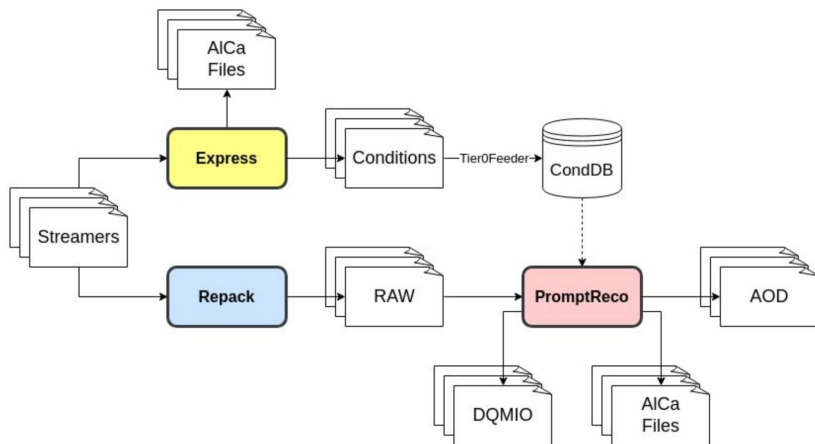
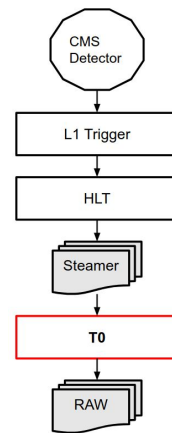
<https://indico.cern.ch/event/1354752/contributions/5704518/attachments/2770289/4829321/T0SystemOverview.pdf>

[https://indico.cern.ch/event/1554074/contributions/6655650/attachments/3137339/5567257/T0%20adaptation%20to%20new%20WM%20](https://indico.cern.ch/event/1554074/contributions/6655650/attachments/3137339/5567257/T0%20adaptation%20to%20new%20WM%20system.pdf)

[0system.pdf](#)

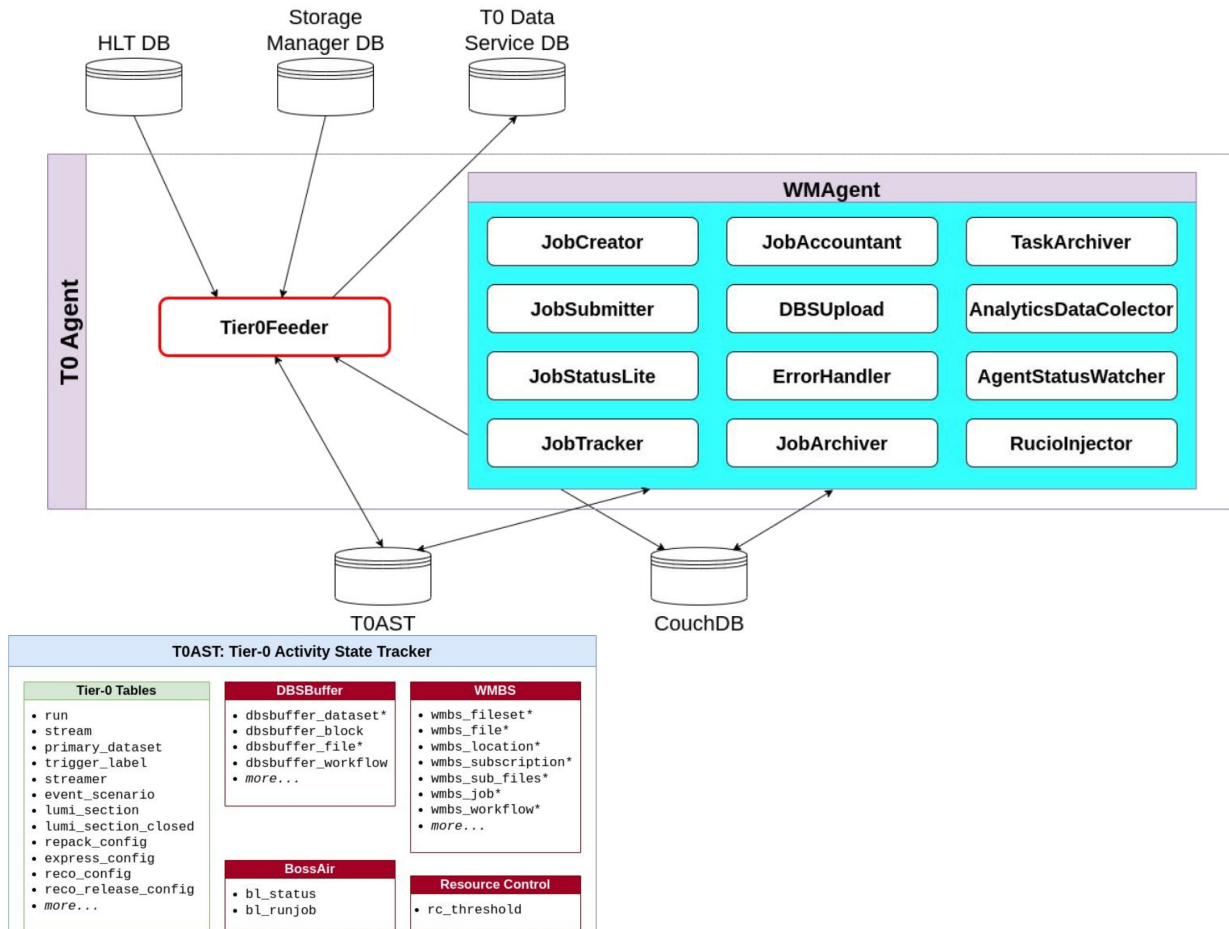
The Tier0 systems and its workflows

- Last step in data acquisition chain (Online) responsible of creating RAW (Online)
- Three types of workflow
 - Repack, Express, PromptReco



- **Repack:** turns release-specific streamer files into forward compatible RAW files for long term storage
 - **Single core, IO intensive jobs.** Typically take 1-8h to finish
- **Express:** quickly reconstruct a small portion of the recorded events in order to determine data taking conditions
 - Short (1h) multi-core jobs
- **PromptReco:** performing the first complete reconstruction of the new data
 - **Multicore, CPU intensive jobs.** Typically take 5-10h to finish

The Tier0 workflow management system



Tier0Feeder: custom component on Tier-0 Agents.

- Inject new data from SM and notifies them about repacking status
- Create Tier-0 workflows in T0AST and CouchDB
- Update information in T0 Data Service
- Uploads conditions to CondDB

Production

cms-wmcore.docs.cern.ch/

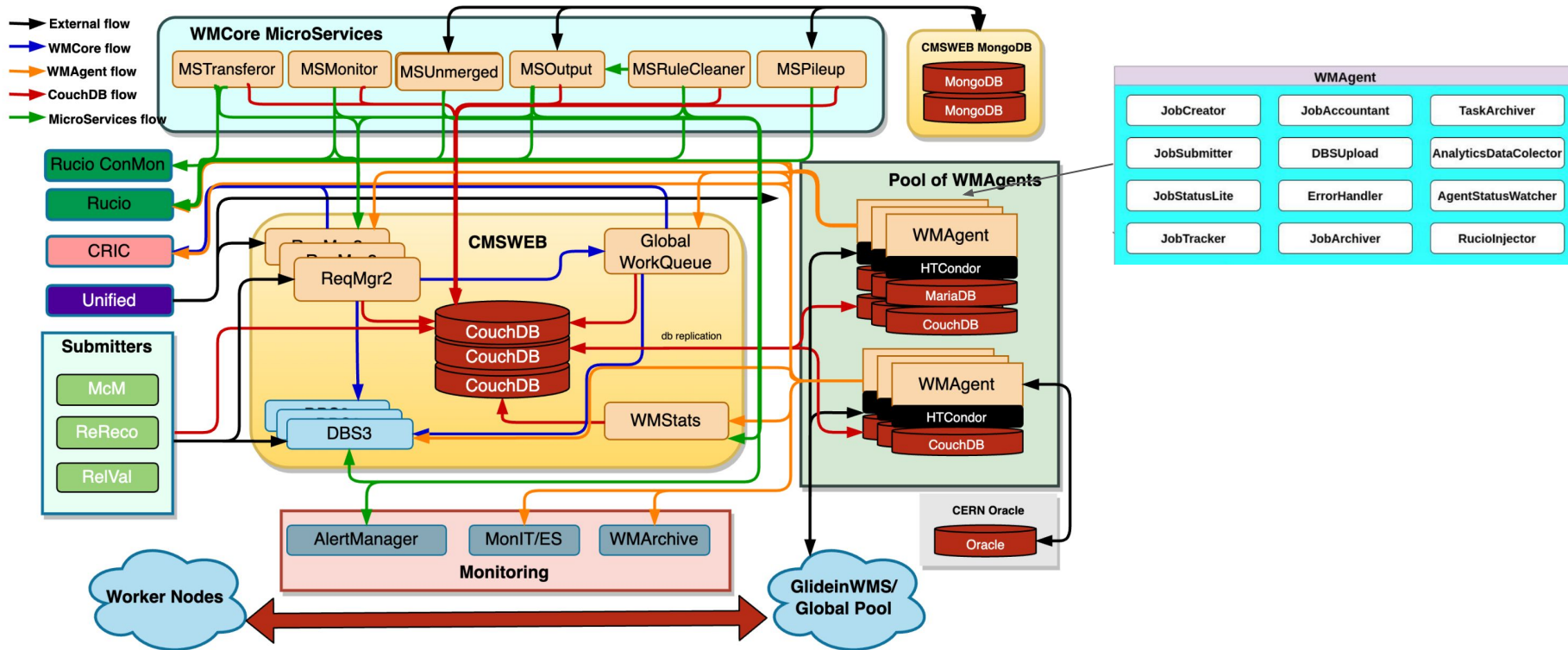
<https://indico.cern.ch/event/1348246/attachments/2754772/4796270/WMAgent%20crash%20course.pdf>

The Production system and its workflows

- Data Reprocessing
- MonteCarlo generation
 - Managed by WMCore and the CompOps team
- Data placement is managed with the help of Rucio
- Workflows:
 - Union of tasks
 - TaskChain: each task is managed as a separate work unit
 - StepChain: the full chain of tasks is managed in the same work unit (the most used)
 - Input data
 - Pileup data, previous data or MC samples to reprocess
 - Intermediate data stay where they are produced
 - Output data
 - Saved with Rucio subscriptions
- Collectively managed by the WMCore Agents and Microservices
 - Operations managed by the CompOps “Unified” tools

The Production workflow management system

CMS Workflow Management System



New WM System

CMS Workflow Management Review 2025 ([link](#))

Context & Scope

- Commissioned by CMS Offline & Computing L1 (Oct 2024); last full review was in 2018
- Covers **WMDM, CRAB/AIS, Submission Infrastructure, Operations**, and related external systems
- Focus: **HL-LHC readiness, sustainability, cost, and operational viability**

Key Findings

- Core workflow components are **15+ years old** and wrapped in increasingly complex operational layers
- **Knowledge concentration risk**: only a few experts understand critical legacy components
- Architecture is **hard to modernize incrementally**; structural limitations persist
- No clear, agreed plan for **user analysis or user-defined simulation workflows** for HL-LHC
- Computing landscape has evolved: **HPC, GPUs, AI/ML, heterogeneous resources**

Lessons Learned

- Past optimism (2018) underestimated **long-term sustainability risks**
- Rucio transition shows value of **shared, externally developed systems**
- Maintaining parallel, independent stacks increases fragility and cost

Strategic Conclusions

- Incremental evolution of the current core is **not sustainable**
- A **major redesign or partial replacement** using external/shared systems is likely required
- Future CMS workflow tools should focus primarily on **HTC workflows**, with separate efforts for:
 - Analysis facilities
 - Tightly coupled / HPC / accelerator workflows

Recommendations

- Clearly document **Run-4 requirements and pain points**
- Evaluate **reuse of components from other workflow systems**
- Define a **realistic transition plan** (including operational impacts, validation, and commissioning)
- Move toward a **common toolset** for central and user-driven HTC workflows

Where do we stand now?

- Evaluation of Panda and Dirac are ongoing
 - Integration of Rucio (Data Management – DM) with Dirac and Panda
 - Run CMS workflows with Panda and Dirac
 - Database backends
 - Integration of HTCondor/GlideInWMS in Dirac and Panda
- Designing internal solution
 - Mostly interested in replacing WMAgent components and optimize existing microservices within the context of a fully modular and integrated system
 - Adopt as late as possible binding for resources matchmaking
 - Tied to queue management system (next slide)
 - Dynamic data placement
 - Question for today: how much can we delegate to HTCondor?

Queue managing system

- Current limitation: Multiple internal queues in the CMS WMS
 - Internal physics-driven prioritization mechanism that prevents materialization of jobs
 - Request Manager => Global WorkQueue => Local WorkQueue => WMBS => APs
 - It currently performs job splitting, early-binding resource matchmaking depending on the live resource availabilities (<https://indico.cern.ch/event/1476884/#preview:5221613>)
 - HTCondor priority and matchmaking mechanisms inefficient because of early prioritization
- Possible solution
 - Reduce the multiple queues (and databases) to one submitting queue (based on FIFO), materializing HTCondor jobs as soon as possible
 - **Fully** delegate prioritization and resource matchmaking to HTCondor
- Open questions:
 - What is the scalability limit of an HTCondor schedd?
 - Does creating more schedds alleviate scalability concerns?
 - What matchmaking and negotiation capabilities HTCondor can handle?
 - DESIRED_Block vs DESIRED_Sites

- Current limitation:
Fixed to a pilot length of 8 hours and relying on unreliable metrics
 - Made on a data block unit level
- Possible solution
 - Introduce a scouting step to infer the processing rates and resource needs by injecting small work units into the schedds
 - Aggregate workflow tasks based on resource needs (“HybridChain”)
 - Automatic splitting algorithm managed via DAGs and SubDAGs
 - CRAB example <https://www.osti.gov/servlets/purl/1420914>
- Open questions:
 - What job splitting capabilities HTCondor can handle?
 - Which mechanism must be managed by the CMS WMS?
 - Divide workflows on data block or time basis?
 - DAG or job clusters?

Dynamic data placement

- Current limitation: data is prestaged at workflow level
 - Jobs are created with site requirements matching the prestaged data location
 - WMS resource GWQ→LWQ early-binding already inherits these site requirements
 - This leads to job stuck in idle because storage and computing needs don't match
- Possible solutions
 - Make the “immutable” data-depending requirements “mutable”
 - DESIRED_Block vs DESIRED_Sites
 - Once the CMS WMS materializes jobs in the HTCondor schedd, we want to be able to place data where is needed
 - How can we “get informed” from HTCondor when we should intervene?
 - Time limit for a job being idle?
 - Calculate job needs (based on event rate), match with a resource cluster, and finally verify data availability?
 - Edit the job description accordingly?
 - Can we adopt data regionality as a classad?
 - Move intermediate data where needed?

Backup slides

More in details...

